DATA ENGINEERING & CLOUD · LESSON 8 OF 15

Apache Spark and PySpark: Distributed Processing for Large Data

Process data too large for one machine with Apache Spark. Understand lazy evaluation, partitions and shuffles, write DataFrame transformations in PySpark, read the execution plan, and fix the common causes of slow jobs such as skew, small files and unnecessary shuffles.

Gap-fill lesson | ✦ 2026 | Advanced | ⏱ 150 min

By the end of this lesson you can

- Explain Spark's driver, executors, partitions and lazy evaluation
- Write PySpark DataFrame transformations and actions
- Identify shuffles and reduce them
- Diagnose slow jobs from the execution plan and the Spark UI

Datasets used

`superstore_sales.csv`

Why Spark

Pandas works well while data fits in one machine's memory. When it does not, Spark splits the data into **partitions** and processes them in parallel on a cluster of machines. Its DataFrame API looks like pandas or SQL, but the work runs across many workers.

CASE STUDY **The job that ran for six hours**

A nightly job joined three years of transactions to a product table, then aggregated. It took six hours and sometimes failed with memory errors. The fix was not a bigger cluster: it was filtering early, broadcasting the small table, and removing an unnecessary shuffle. The job then ran in twelve minutes.

The architecture

Component	Role
Driver	Plans the work, holds the program, coordinates the executors
Executors	Worker processes that run tasks on partitions and hold cached data
Partition	A chunk of the data processed as one task
Cluster manager	Allocates machines (YARN, Kubernetes, or the managed platform's own)

Lazy evaluation

Spark separates **transformations** (filter, select, join, groupBy) from **actions** (count, show, write). Transformations only build a plan. Nothing runs until an action asks for a result. Spark can then optimise the whole plan, combining steps and skipping work that is not needed.

Python

```python
# Illustrative PySpark (not run: needs a Spark installation). Shows the pattern.
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("sales_summary").getOrCreate()

sales = spark.read.option("header", True).option("inferSchema",
True).csv("superstore_sales.csv")
products = spark.read.option("header", True).csv("products.csv")

summary = (
    sales
    .filter(F.col("order_date") ≥ "2024-01-01")          # transformation: filter early
    .join(F.broadcast(products), "product_id")           # broadcast the small table
    .groupBy("category", F.date_trunc("month", "order_date").alias("month"))
    .agg(F.sum("sales").alias("sales"), F.sum("profit").alias("profit"))
)

summary.explain()                                         # show the plan, not yet
executed
summary.write.mode("overwrite").parquet("gold/monthly_category_sales")   # action: runs the
job
```

`explain()` prints the plan Spark will run. `write` is the action that starts the work. Read the plan before you tune anything.

Shuffles: the expensive step

A **shuffle** moves data between executors so that rows with the same key end up together. Joins on non-broadcast tables and `groupBy` both shuffle. Shuffles write to disk and send data over the network, which is usually the slowest part of a job.

Operation	Shuffle?	Cheaper alternative
Filter, select, map	No	–
`groupBy` with aggregation	Yes, but Spark pre-aggregates first	Keep aggregations simple
Join with a large table	Yes, both sides	Partition both tables on the join key in advance
Join with a small table	Avoidable	`broadcast(small_table)`
Repeated sorting for each step	Often	Sort once, or avoid sorting

::::tip Broadcast the small table A join with a table that fits in memory can send a copy to every executor, so the big table never has to shuffle. Use `F.broadcast()` on the small side. Spark also does this automatically below a size threshold, but explicit hints make the intent clear. ::::

Partitions: how many and how big

- Too few partitions leave executors idle; too many add scheduling overhead and tiny files.
- A common starting point is a few partitions per core, and partitions of roughly 100 to 200 MB.
- `repartition(n)` changes the number of partitions with a shuffle; `coalesce(n)` reduces it without one.

Data skew

Skew happens when one key holds most of the rows (for example, one customer id with a million orders). The task for that key runs far longer than the others, and the job waits for it. Diagnose skew in the Spark UI: one task takes much longer than its siblings. Fixes include filtering that key separately, salting the key, or aggregating in two stages.

Small files

Writing many tiny files slows every later read, because each file has overhead. Compact them with `coalesce` or the platform's compaction feature before writing the output for downstream use.

Reading the Spark UI

Tab	Look for
Jobs and stages	Which stage is slow and how many tasks it has
Stage details	Task time spread: one long task suggests skew
SQL tab	The physical plan with its shuffle and join types
Storage	Cached data and how much memory it uses

Caching

`cache()` or `persist()` keeps a DataFrame in memory when you use it more than once. Cache after expensive steps that feed several outputs, and unpersist when you finish. Caching everything wastes memory and can slow the job.

INTERVIEW CORNER **Why is a Spark job slow, and how would you find out?**

Read the plan with `explain()`, open the Spark UI, and find the slowest stage. Typical causes are a large unbroadcast join, a shuffle that could be avoided by filtering earlier, skewed keys where one task runs far longer, and many small files. Fix the largest cost first and measure again.

::::key What to remember - Spark processes partitions in parallel; transformations are lazy and actions start the work. - Shuffles are the expensive step: filter early, broadcast small tables, and avoid unnecessary sorts. - Choose partitions of about 100–200 MB and avoid tiny files. - Diagnose skew in the Spark UI and fix it with separate handling or two-stage aggregation. - Read the plan and the UI before tuning. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which operation starts the work in Spark?

A filter

B select

C write (an action)

D withColumn

Answer: C. Actions trigger execution; transformations only build the plan.

2. What is the main cost in a Spark job's slow step?

A Writing a comment

B Shuffling data between executors

C Printing the schema

D Reading the header row

Answer: B. Shuffles move data across the network and disk.

3. How do you avoid a shuffle when joining a large table with a small lookup table?

A Repartition both tables to one partition

B Broadcast the small table

C Sort both tables

D Cache the output

Answer: B. A broadcast join sends the small table to each executor.

4. One task in a stage takes 40 minutes while the others take 1 minute. What is the likely problem?

A The cluster has too many machines

B Data skew: one key holds much more data than the others

C The file has a header

D The output is in Parquet

Answer: B. One long task among many short ones is the signature of skew.

5. Why avoid writing thousands of tiny files?

A They cannot be read

B Each file adds overhead, so later reads become slow

C Spark does not support files

D They cost nothing

Answer: B. Compaction reduces the overhead for downstream reads.

EXERCISE **Tune a slow plan**

Write, in pseudocode or PySpark, a job that joins a 500-million-row transactions table to a 2,000-row product table and aggregates by category. List three changes you would make to reduce the cost, the expected effect of each, and how you would confirm the improvement with `explain()` and the Spark UI.

Show solution (worked answer)

Expected changes: broadcast the product table; filter to the needed date range before the join; avoid a second shuffle by aggregating once with the right keys. Confirm by comparing the plan's join type and the stage times in the UI.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.